

MODULE-2:

JDBC (Java Database Connectivity)

- **What is JDBC?** JDBC is an API (Application Programming Interface) in Java that allows Java applications to interact with databases. It provides classes and interfaces to connect, execute queries, and retrieve results from relational databases like MySQL, Oracle, etc.
- **Importance of JDBC in Java Programming**
 - Enables seamless database connectivity.
 - Provides a standard way to perform CRUD (Create, Read, Update, Delete) operations.
 - Makes applications dynamic by storing and retrieving data.
 - Supports multiple databases through different drivers.
- **JDBC Architecture Components**
 - **Driver Manager:** Manages database drivers and establishes connections.
 - **Driver:** Translates JDBC calls into database-specific calls.
 - **Connection:** Represents a session with the database.
 - **Statement:** Used to execute SQL queries.
 - **ResultSet:** Represents the result of a query, allowing traversal of rows.

Lab Exercise

Program: Connect to MySQL using JDBC

java

Copy

```
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.Statement;  
import java.sql.ResultSet;
```

```
public class JdbcExample {  
    public static void main(String[] args) {  
        try {  
            // Load JDBC driver  
            Class.forName("com.mysql.cj.jdbc.Driver");
```

```

// Establish connection
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/testdb", "root", "password");

// Create statement
Statement stmt = con.createStatement();

// Execute query
ResultSet rs = stmt.executeQuery("SELECT * FROM students");

// Process results
while (rs.next()) {
    System.out.println(rs.getInt("id") + " " + rs.getString("name"));
}

// Close connection
con.close();
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

Steps for Creating JDBC Connections • Theory: o Step-by-Step Process to Establish a JDBC Connection:

1. Import the JDBC packages
2. Register the JDBC driver
3. Open a connection to the database
4. Create a statement
5. Execute SQL queries
6. Process the result set
7. Close the connection

Steps to Establish a JDBC Connection

1. Import the JDBC packages

- Use `import java.sql.*;` to access JDBC classes and interfaces.

2. Register the JDBC driver

- Load the driver class using `Class.forName("com.mysql.cj.jdbc.Driver");`.

3. Open a connection to the database

- Use `DriverManager.getConnection(url, user, password);`.

4. Create a statement

- Use `Connection.createStatement()` or `Connection.prepareStatement()`.

5. Execute SQL queries

- Use `executeQuery()` for SELECT, `executeUpdate()` for INSERT/UPDATE/DELETE.

6. Process the result set

- Traverse results using `ResultSet` methods like `next()`, `getString()`, `getInt()`

7. Close the connection

- Always close `ResultSet`, `Statement`, and `Connection` to free resources.

```
import java.sql.Connection;
```

```
import java.sql.DriverManager;
```

```
public class JdbcConnectionDemo {
    public static void main(String[] args) {
        try {
            // Step 2: Register JDBC driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Step 3: Open a connection
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb", "root", "password");

            // Confirmation message
            if (con != null) {
                System.out.println("✅ Connection established successfully!");
            }

            // Step 7: Close the connection
            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Theory: Types of JDBC Statements

- **Statement**
 - Executes simple SQL queries without parameters.
 - Suitable for static queries.
 - Example: `stmt.executeQuery("SELECT * FROM students");`
- **PreparedStatement**
 - Precompiled SQL statement.
 - Allows parameterized queries (placeholders ?).
 - More secure (prevents SQL injection)
 - `?`
 - and efficient for repeated queries.
 - Example:
 - java
 - `PreparedStatement ps = con.prepareStatement("SELECT * FROM students WHERE id=?");`
 - `ps.setInt(1, 101);`
- **CallableStatement**
 - Used to call stored procedures in the database.
 - Supports input, output, and in-out parameters.

```
CallableStatement cs = con.prepareCall("{call getStudentName(?) }");  
cs.setInt(1, 101);
```

Differences Between Statement, PreparedStatement, and CallableStatement

Feature	Statement	PreparedStatement	CallableStatement
Use Case	Simple queries	Parameterized queries	Stored procedures
Efficiency	Less efficient	More efficient (precompiled)	Efficient for procedures

Feature	Statement	PreparedStatement	CallableStatement
Security	Vulnerable to SQL injection	Prevents SQL injection	Secure
Flexibility	Static SQL only	Dynamic with parameters	

```
import java.sql.*;

import java.sql.*;

public class StatementCRUD {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb", "root", "password");
            Statement stmt = con.createStatement();

            // Insert
            stmt.executeUpdate("INSERT INTO students VALUES (101, 'Priya')");

            // Update
            stmt.executeUpdate("UPDATE students SET name='Priyanka' WHERE id=101");

            // Select
            ResultSet rs = stmt.executeQuery("SELECT * FROM students");
            while (rs.next()) {
                System.out.println(rs.getInt("id") + " " + rs.getString("name"));
            }

            // Delete
            stmt.executeUpdate("DELETE FROM students WHERE id=101");

            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Using PreparedStatement for Parameterized Queries

```

import java.sql.*;

public class PreparedStatementDemo {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb", "root", "password");

            // Insert with parameters
            PreparedStatement ps = con.prepareStatement("INSERT INTO students VALUES (?, ?)");
            ps.setInt(1, 102);
            ps.setString(2, "Rahul");
            ps.executeUpdate();

            // Select with parameters
            ps = con.prepareStatement("SELECT * FROM students WHERE id=?");
            ps.setInt(1, 102);
            ResultSet rs = ps.executeQuery();
            while (rs.next()) {
                System.out.println(rs.getInt("id") + " " + rs.getString("name"));
            }

            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Theory: JDBC CRUD Operations

- **Insert**
 - Adds a new record into the database table.
 - Example SQL: INSERT INTO students VALUES (101, 'Priya');
- **Update**
 - Modifies existing records in the table.
 - Example SQL: UPDATE students SET name='Priyanka' WHERE id=101;
- **Select**
 - Retrieves records from the database.
 - Example SQL: SELECT * FROM students;
- **Delete**
 - Removes records from the database.
 - Example SQL: DELETE FROM students WHERE id=101;

JDBC CRUD Program:

```

import java.sql.*;

public class JdbcCRUD {
    public static void main(String[] args) {
        try {

```

```

// Step 1: Load JDBC driver
Class.forName("com.mysql.cj.jdbc.Driver");

// Step 2: Establish connection
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/testdb", "root", "password");

Statement stmt = con.createStatement();

// INSERT
stmt.executeUpdate("INSERT INTO students VALUES (101, 'Priya')");
System.out.println("✅ Record inserted successfully");

// UPDATE
stmt.executeUpdate("UPDATE students SET name='Priyanka' WHERE id=101");
System.out.println("✅ Record updated successfully");

// SELECT
ResultSet rs = stmt.executeQuery("SELECT * FROM students");
System.out.println("✅ Records in table:");
while (rs.next()) {
    System.out.println(rs.getInt("id") + " " + rs.getString("name"));
}

// DELETE
stmt.executeUpdate("DELETE FROM students WHERE id=101");
System.out.println("✅ Record deleted successfully");

// Close connection
con.close();
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

